SOP PROGRESS REPORT

MONTH 4

1.Collection and Organization of the Simulated data

With the mathematical model and PID-based control system established in the previous month, the next step was to obtain and organise the numerical data generated during the drone simulation. The MATLAB simulation was modified to systematically record the state variables and control inputs at every simulation time step, alongside the real-time 3D visualisation of the quadcopter.

The acquired data was organised into 17 individual time-history plots, allowing each important parameter of the drone and its control system to be studied separately. These plots were generated simultaneously with the drone simulation and provide a quantitative representation of the drone's behaviour throughout the flight.

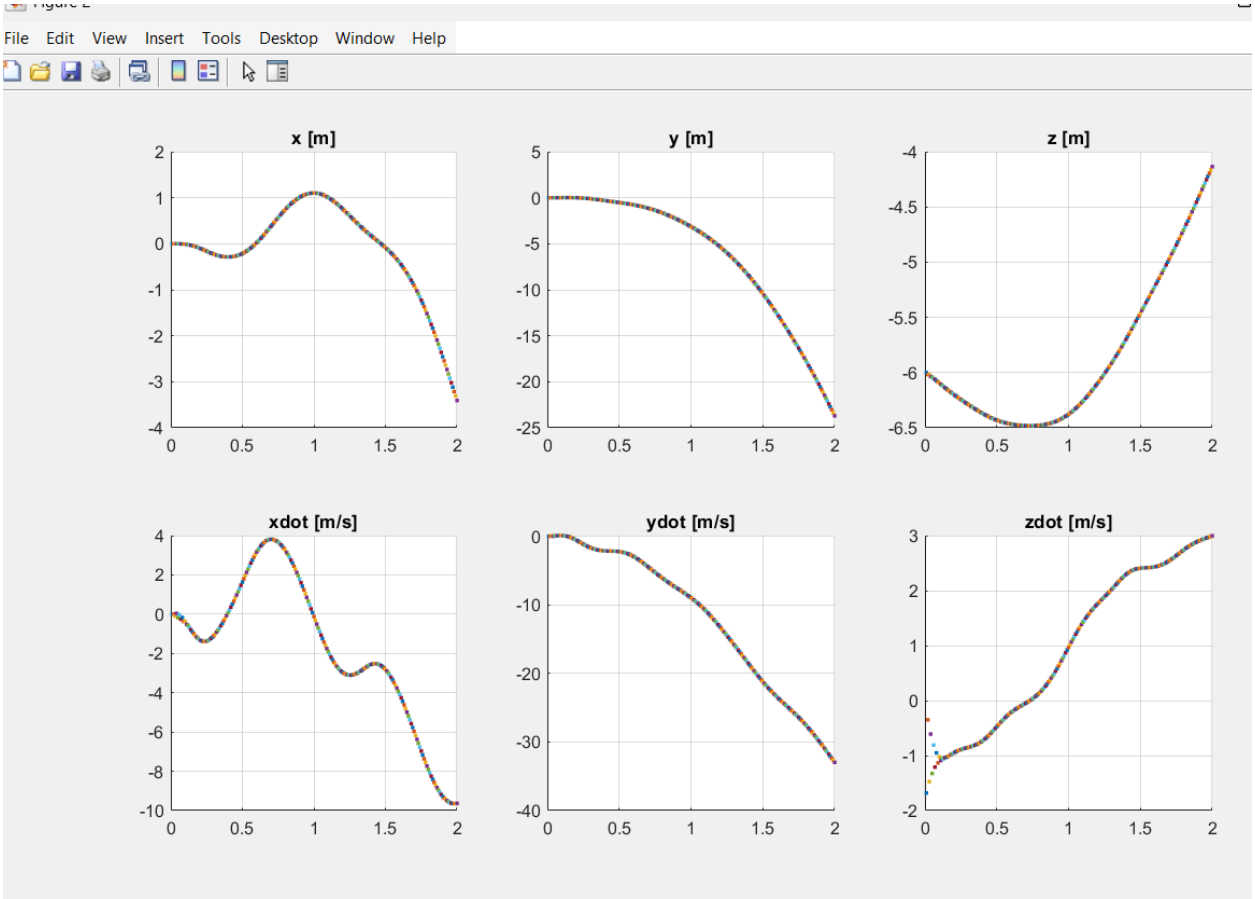
The 17 plotted quantities were organised as follows:

Position

1. x-position
2. y-position
3. z-position

Linear velocity

4. x_dot
5. y_dot
6. z_dot

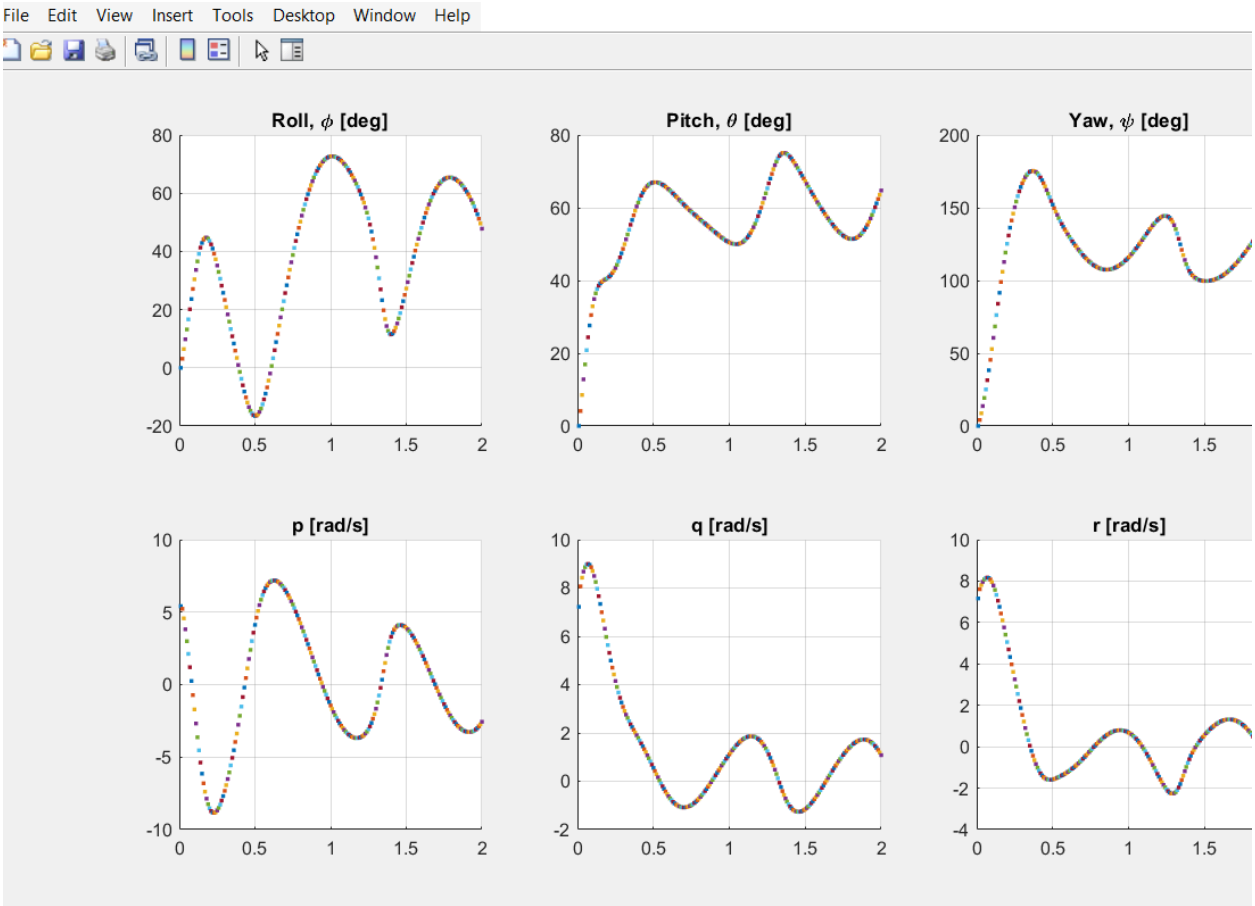


Orientation

7. Roll (ϕ)
8. Pitch (θ)
9. Yaw (ψ)

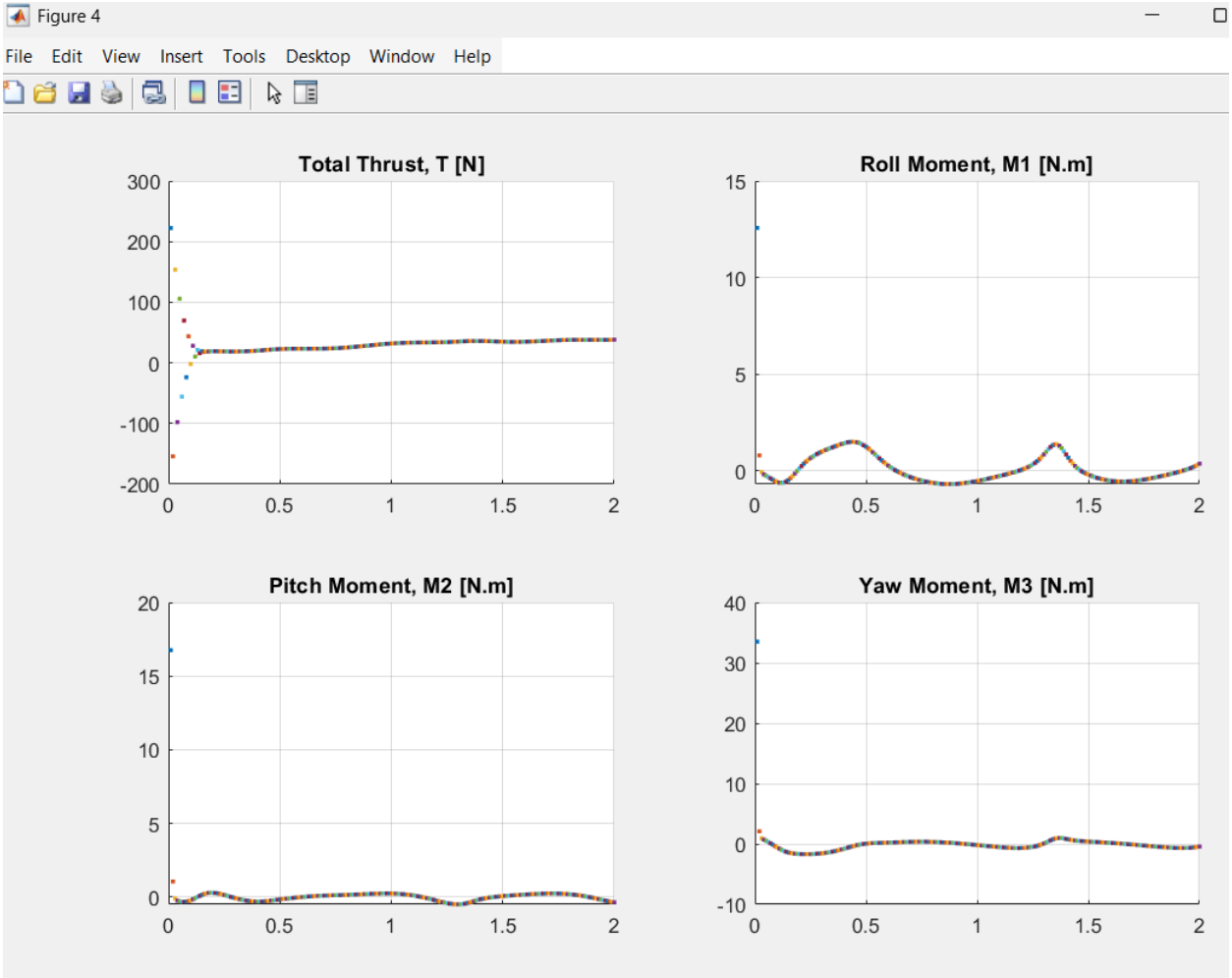
Angular velocity

10. p — angular velocity about the body x-axis
11. q — angular velocity about the body y-axis
12. r — angular velocity about the body z-axis



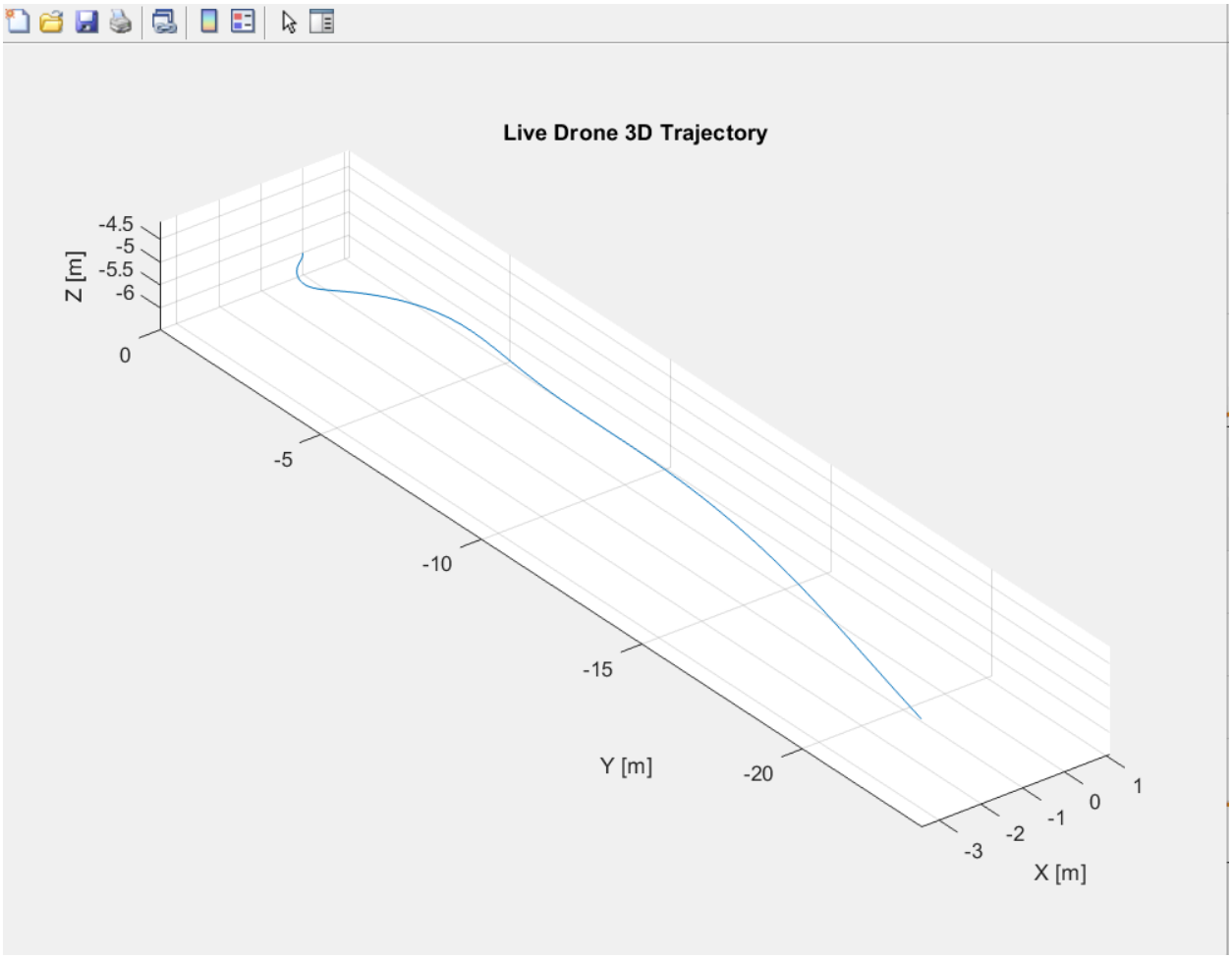
Controller outputs

13. Thrust T
14. Roll moment M1
15. Pitch moment M2
16. Yaw moment M3



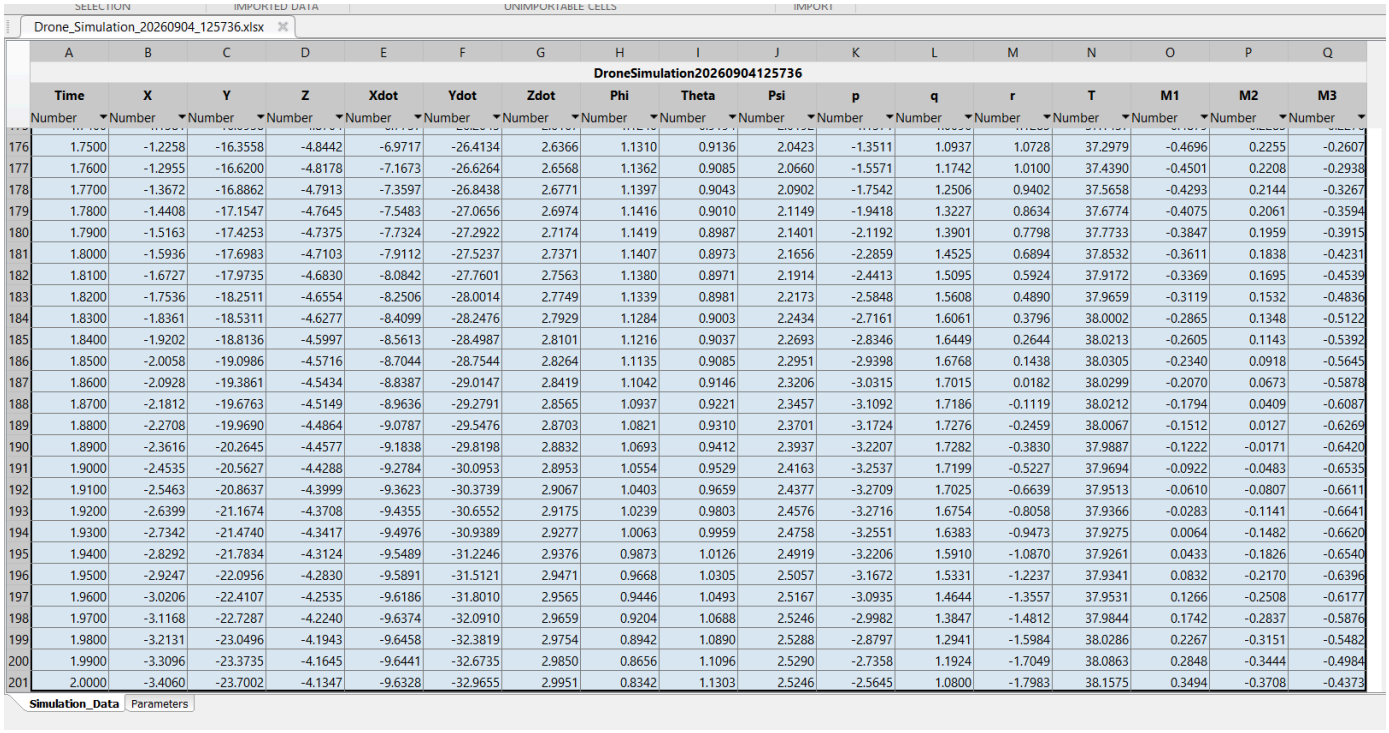
Trajectory

17. Live 3D Trajectory



2. Export of Simulation Results to Excel

To facilitate further analysis and maintain a structured record of the simulation results, an automated Excel export facility was incorporated into the MATLAB simulation. Instead of relying only on the graphs generated during the simulation, the numerical values obtained at every simulation time step are now stored and exported as a structured Excel file. The data that are compiled using the excel sheet can be used for training drones for disaster search and rescue.



Drone_Simulation_20260904_125736.xlsx

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
	DroneSimulation20260904125736																
	Time	X	Y	Z	Xdot	Ydot	Zdot	Phi	Theta	Psi	p	q	r	T	M1	M2	M3
	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number	Number
176	1.7500	-1.2258	-16.3558	-4.8442	-6.9717	-26.4134	2.6366	1.1310	0.9136	2.0423	-1.3511	1.0937	1.0728	37.2979	-0.4696	0.2255	-0.2607
177	1.7600	-1.2955	-16.6200	-4.8178	-7.1673	-26.6264	2.6568	1.1362	0.9085	2.0660	-1.5571	1.1742	1.0100	37.4390	-0.4501	0.2208	-0.2938
178	1.7700	-1.3672	-16.8862	-4.7913	-7.3597	-26.8438	2.6771	1.1397	0.9043	2.0902	-1.7542	1.2506	0.9402	37.5658	-0.4293	0.2144	-0.3267
179	1.7800	-1.4408	-17.1547	-4.7645	-7.5483	-27.0656	2.6974	1.1416	0.9010	2.1149	-1.9418	1.3227	0.8634	37.6774	-0.4075	0.2061	-0.3594
180	1.7900	-1.5163	-17.4253	-4.7375	-7.7324	-27.2922	2.7174	1.1419	0.8987	2.1401	-2.1192	1.3901	0.7798	37.7733	-0.3847	0.1959	-0.3915
181	1.8000	-1.5936	-17.6983	-4.7103	-7.9112	-27.5237	2.7371	1.1407	0.8973	2.1656	-2.2859	1.4525	0.6894	37.8532	-0.3611	0.1838	-0.4231
182	1.8100	-1.6727	-17.9735	-4.6830	-8.0842	-27.7601	2.7563	1.1380	0.8971	2.1914	-2.4413	1.5095	0.5924	37.9172	-0.3369	0.1695	-0.4539
183	1.8200	-1.7536	-18.2511	-4.6554	-8.2506	-28.0014	2.7749	1.1339	0.8981	2.2173	-2.5848	1.5608	0.4890	37.9659	-0.3119	0.1532	-0.4836
184	1.8300	-1.8361	-18.5311	-4.6277	-8.4099	-28.2476	2.7929	1.1284	0.9003	2.2434	-2.7161	1.6061	0.3796	38.0002	-0.2865	0.1348	-0.5122
185	1.8400	-1.9202	-18.8136	-4.5997	-8.5613	-28.4987	2.8101	1.1216	0.9037	2.2693	-2.8346	1.6449	0.2644	38.0213	-0.2605	0.1143	-0.5392
186	1.8500	-2.0058	-19.0986	-4.5716	-8.7044	-28.7544	2.8264	1.1135	0.9085	2.2951	-2.9398	1.6768	0.1438	38.0305	-0.2340	0.0918	-0.5645
187	1.8600	-2.0928	-19.3861	-4.5434	-8.8387	-29.0147	2.8419	1.1042	0.9146	2.3206	-3.0315	1.7015	0.0182	38.0299	-0.2070	0.0673	-0.5878
188	1.8700	-2.1812	-19.6763	-4.5149	-8.9636	-29.2791	2.8565	1.0937	0.9221	2.3457	-3.1092	1.7186	-0.1119	38.0212	-0.1794	0.0409	-0.6087
189	1.8800	-2.2708	-19.9690	-4.4864	-9.0787	-29.5476	2.8703	1.0821	0.9310	2.3701	-3.1724	1.7276	-0.2459	38.0067	-0.1512	0.0127	-0.6269
190	1.8900	-2.3616	-20.2645	-4.4577	-9.1838	-29.8198	2.8832	1.0693	0.9412	2.3937	-3.2207	1.7282	-0.3830	37.9887	-0.1222	-0.0171	-0.6420
191	1.9000	-2.4535	-20.5627	-4.4288	-9.2784	-30.0953	2.8953	1.0554	0.9529	2.4163	-3.2537	1.7199	-0.5227	37.9694	-0.0922	-0.0483	-0.6535
192	1.9100	-2.5463	-20.8637	-4.3999	-9.3623	-30.3739	2.9067	1.0403	0.9659	2.4377	-3.2709	1.7025	-0.6639	37.9513	-0.0610	-0.0807	-0.6611
193	1.9200	-2.6399	-21.1674	-4.3708	-9.4355	-30.6552	2.9175	1.0239	0.9803	2.4576	-3.2716	1.6754	-0.8058	37.9366	-0.0283	-0.1141	-0.6641
194	1.9300	-2.7342	-21.4740	-4.3417	-9.4976	-30.9389	2.9277	1.0063	0.9959	2.4758	-3.2551	1.6383	-0.9473	37.9275	0.0064	-0.1482	-0.6620
195	1.9400	-2.8292	-21.7834	-4.3124	-9.5489	-31.2246	2.9376	0.9873	1.0126	2.4919	-3.2206	1.5910	-1.0870	37.9261	0.0433	-0.1826	-0.6540
196	1.9500	-2.9247	-22.0956	-4.2830	-9.5891	-31.5121	2.9471	0.9668	1.0305	2.5057	-3.1672	1.5331	-1.2237	37.9341	0.0832	-0.2170	-0.6396
197	1.9600	-3.0206	-22.4107	-4.2535	-9.6186	-31.8010	2.9565	0.9446	1.0493	2.5167	-3.0935	1.4644	-1.3557	37.9531	0.1266	-0.2508	-0.6177
198	1.9700	-3.1168	-22.7287	-4.2240	-9.6374	-32.0910	2.9659	0.9204	1.0688	2.5246	-2.9982	1.3847	-1.4812	37.9844	0.1742	-0.2837	-0.5876
199	1.9800	-3.2131	-23.0496	-4.1943	-9.6458	-32.3819	2.9754	0.8942	1.0890	2.5288	-2.8797	1.2941	-1.5984	38.0286	0.2267	-0.3151	-0.5482
200	1.9900	-3.3096	-23.3735	-4.1645	-9.6441	-32.6735	2.9850	0.8656	1.1096	2.5290	-2.7358	1.1924	-1.7049	38.0863	0.2848	-0.3444	-0.4984
201	2.0000	-3.4060	-23.7002	-4.1347	-9.6328	-32.9655	2.9951	0.8342	1.1303	2.5246	-2.5645	1.0800	-1.7983	38.1575	0.3494	-0.3708	-0.4373

Simulation_Data | Parameters

Parameters tab contains the physical drone parameters, gravitational constant, simulation settings and PID gains used for that particular simulation run. This makes each generated dataset self-contained and allows different simulation runs to be reproduced and compared based on their corresponding parameter sets.

3. Introduction to Filters

In the existing drone simulation, the mathematical model provides the exact state of the drone at every instant. However, in a real drone, these states cannot be known perfectly. They have to be obtained from sensors such as IMU, GPS, barometer, cameras, etc., and these measurements are affected by noise and uncertainty.

This creates two sources of information about the drone's state:

1. **Model prediction** – The mathematical model predicts where the drone should be based on its previous state, dynamics and control inputs.
2. **Sensor measurement** – Sensors provide measurements of the actual drone, but these measurements contain noise and may not exactly represent the true state.

Now, we are planning to use two filters : Particle and Kalmann filters

They use their own methodologies to determine the most probable state of the drone.

The **Kalman Filter** represents the state using a probability distribution, generally described by a mean and covariance. It predicts the state using the mathematical model and then updates this prediction using the sensor measurement. The Kalman Gain determines how strongly the new measurement should influence the predicted state.

The **Particle Filter**, on the other hand, does not represent the uncertainty using a single mean and covariance. It represents possible states using a large number of particles, with each particle representing a possible state of the system. Sensor measurements are then used to assign greater or smaller weights to these possible states, followed by resampling.

As part of the learning process, I also explored a practical example of a Particle Filter being used for LiDAR-based localisation of a mobile robot. Observing how multiple possible particle states were updated using sensor observations helped me understand how particle filters handle uncertainty and how their application differs between robot localisation and the state-estimation problem in a drone.

4. Reading a Research Paper on GNSS-denied UAV–UGV search-and-rescue

1. GNSS cannot be assumed to always be available

The central problem is:

What happens when a UAV/UGV cannot reliably obtain a global position?

The project specifically investigates operations where globally reliable positioning cannot be assumed. This is important for our project because our current MATLAB drone assumes that its state is perfectly known. A real autonomous drone cannot make that assumption.

So it reinforced our motivation for introducing state estimation.

2. The drone has to estimate its state from sensors

The research uses different sources of sensor information rather than relying on a single perfect position measurement.

Some of the sensors used for the self detection

- IMU
- wheel odometry
- LiDAR

and combines these to obtain local state information.

3. Local Maps and Relative Position Estimation

Another aspect we learnt from the research was how multiple robots can operate in a GNSS-denied environment without sharing a common global coordinate system.

Each robot independently uses its onboard sensors to construct its own localised map of the environment. Since the UAV and UGV are operating independently, their maps initially exist in their own local coordinate frames.

The two maps therefore do not automatically have the same origin, orientation or coordinate axes.

To determine the relative position between the robots, a transformation between the two local map frames is established. This transformation relates coordinates expressed in one robot's map to coordinates expressed in the other's map.

Once this transformation is known, a position identified in the UAV's local coordinate frame can be expressed in the UGV's frame, and vice versa. This allows the system to determine the relative position and orientation of the two robots, even without a common GNSS-based global reference.

4. Uncertainty-Aware State Estimation and Communication

A further concept we learnt from the research was that state estimation is inherently uncertain. Even after sensor fusion and localization, the estimated position of a robot may differ from its actual position. Therefore, the system continuously evaluates the uncertainty or error associated with its estimated state.

This uncertainty can then be used to decide when communication between the robots is actually necessary. Instead of continuously transmitting localization and map information, which can consume communication bandwidth, a robot can communicate when the uncertainty in its estimated state becomes sufficiently large or when new information is important for the other robot.

The research paper provided a broader perspective on how the state-estimation framework being developed in our project can be extended toward real autonomous systems. It demonstrated the importance of sensor fusion, uncertainty estimation, local coordinate frames and uncertainty-aware communication in GNSS-denied environments. For our current project, these concepts motivate the transition from an ideal simulation with perfect state information toward a more realistic system based on noisy measurements and estimated states.